Q

QS-AMP

Quantum Security Assessment & Migration Platform

Engineering Walkthrough Guide

What it is, how it is built, and what every screen does - written for hand-off and TL review.

React 19 + Vite | Django 5 + DRF | PostgreSQL | JWT Auth | CycloneDX CBOM | ML-KEM / ML-DSA / SLH-DSA

Table of Contents

1. What Is QS-AMP 4

Why this exists (the problem) . . . 4

Who uses it, and the two states of the app . . . 4

2. Architecture & Tech Stack 6

Request flow . . . 6

Project layout . . . 7

3. End-to-End: What Happens When You Add a Source 8

Worked example (the demo data set) . . . 8

4. Onboarding & Adding a Source 9

5. Module-by-Module Walkthrough 10

5.1 Overview (Dashboard) 11

5.2 Discovery 12

5.3 Crypto Inventory 13

5.4 CBOM Explorer 14

5.5 Risk Assessment 15

5.6 PQC Comparison 16

5.7 Dependencies (Architecture graph) 17

5.8 Monitoring 18

5.9 Migration Planner 19

5.10 Benchmarks 20

5.11 Reports 21

5.12 Alerts 22

5.13 Settings 23

6. What Is Real vs. Simulated - Be Ready for This Question 24

7. Security & Correctness Highlights 25

Security controls 25

Cryptographic correctness (the part most tools get wrong) 25

8. Quick Answers for Common Questions 26

1. What Is QS-AMP

QS-AMP is an internal web application that answers one question for a security team: **"Where does our software use cryptography, and how exposed is it once large quantum computers exist?"** It scans real sources (websites, ZIP uploads, public Git repositories), records every algorithm/key/certificate/protocol it finds as a structured inventory, scores each finding for quantum risk, recommends the correct post-quantum replacement for that specific use, and tracks the migration work through to completion.

It is built as a full-stack product, not a slide deck: a Django REST API backed by PostgreSQL, a React single-page app, real TLS probing, a real (if unoptimised) ML-KEM/ML-DSA benchmark, and a CycloneDX 1.6 CBOM (Cryptography Bill of Materials) export - the same industry format NIST and OWASP use for crypto inventories.

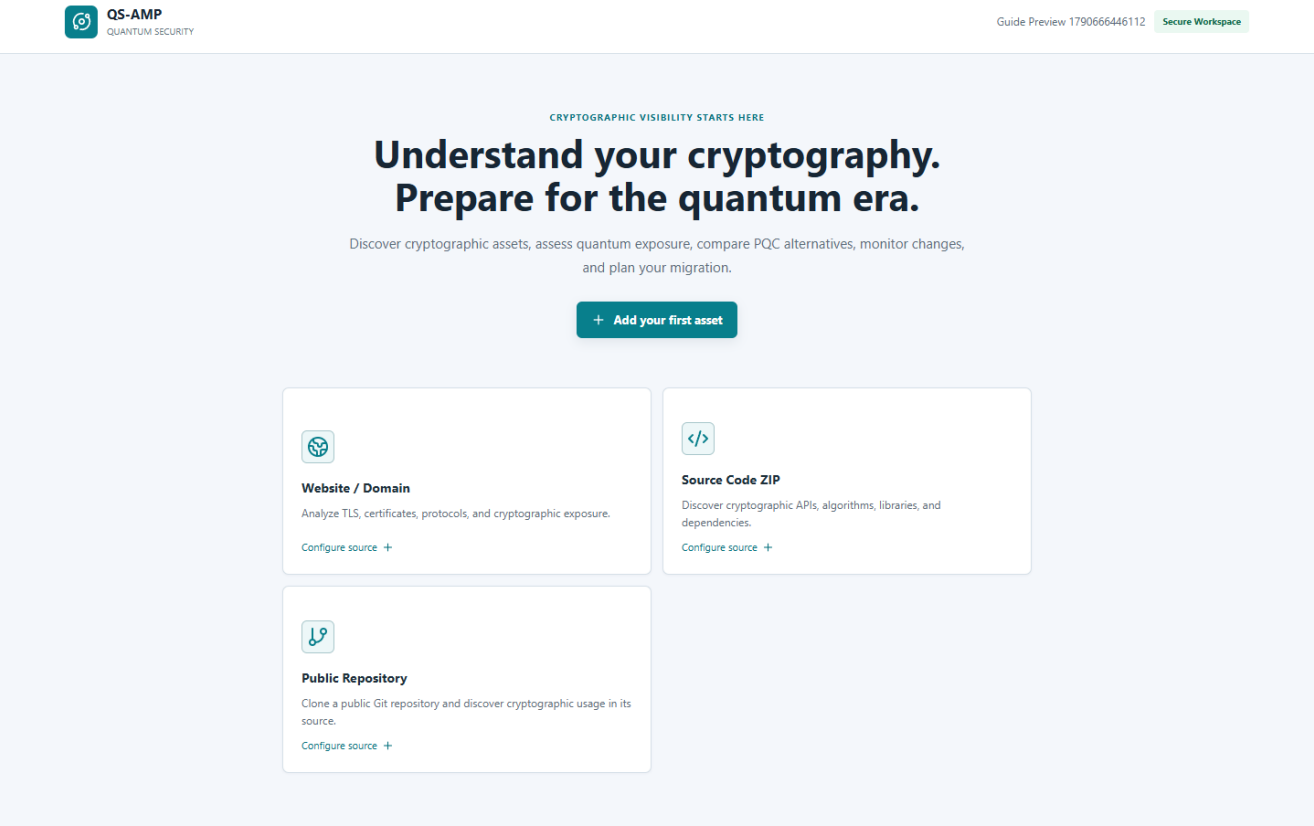
Why this exists (the problem)

- **"Harvest now, decrypt later"**: an attacker can record TLS/VPN traffic today and decrypt it once a cryptographically-relevant quantum computer exists. Anything protecting long-lived confidential data (RSA/ECDH key exchange today) is exposed retroactively, not just going forward.
- NIST finalised the first post-quantum standards in August 2024 (FIPS 203 ML-KEM, FIPS 204 ML-DSA, FIPS 205 SLH-DSA). Organisations now need to know *where* RSA/ECDSA/ECDH are actually used before they can plan a migration - most have no such inventory.
- Migrating blindly is dangerous: a KEM (key exchange) is not a signature algorithm. QS-AMP's core design rule is that it never recommends a replacement without first knowing the *purpose* the classical algorithm serves.

Who uses it, and the two states of the app

The intended user is a security engineer or architect. The app has exactly two states, by design:

- **State 1 - empty workspace.** A brand-new organisation sees only a landing page: a headline, an explanation, and an "Add your first asset" button. There is no sidebar and nothing to configure yet.
- **State 2 - active workspace.** The moment one source is added and its first scan completes, the full 13-item sidebar (Overview through Settings) appears and stays available from then on.



State 1 - the empty workspace a brand-new organisation sees (root route, no sources yet).

2. Architecture & Tech Stack

QS-AMP is a conventional three-tier web app. Nothing exotic, which is deliberate - every piece can be explained and defended in a design review.

Layer	Technology	Notes
Frontend	React 19 + Vite 7	Single-page app, plain CSS (no Tailwind/CSS framework), React Router 7 for routing.
Charts	Recharts 3	All dashboard bar/line/donut charts.
Graphs	@xyflow/react (React Flow)	CBOM component graph and the crypto dependency map are real interactive node graphs, not images.
Motion	Framer Motion 12	Page transitions, card entrances, the scan-progress animation.
Icons	lucide-react	Every icon in the app.
Uploads	react-dropzone	Drag-and-drop for ZIP and CBOM file uploads.
API layer	Django 5 + Django REST Framework	One Django "app" per domain (see section 5); ViewSets + serializers + django-filter.
Auth	djangorestframework-simplejwt + pyotp	JWT access/refresh tokens; optional TOTP-based MFA; hashed API keys as an alternative to JWT.
Database	PostgreSQL	Every organisation-owned table carries an explicit organization_id foreign key and is filtered through it on every query (see section 6).
Background work	Celery + Redis (optional)	Scans and monitoring runs work inline via a worker thread in dev; the same code path is Celery-ready for production.
Reports	ReportLab	Generates the actual PDF files served from /api/reports/.
PQC libraries	kyber-py, dilithium-py (pure Python reference impls)	Used for real ML-KEM / ML-DSA benchmark timings; architecture also has an adapter slot for native liboqs.

Request flow

Browser (React) -> Axios client (attaches JWT + active-organization header) -> Django URL routing (`/api/<module>/`) -> DRF permission check (role + organisation membership) -> a *service* module holds the actual business logic (never the view) -> PostgreSQL. Every write also drops a row in the audit log.

Project layout

```
backend/apps/   accounts - organizations - discovery - inventory - cbom - risk - comparison - benchmarks - monitoring - migration - reports - alerts - integrations - audit
frontend/src/ pages (one file per screen) - components (shared UI: DataTable, drawers, badges) - api (Axios client) - context (auth/session)
```

3. End-to-End: What Happens When You Add a Source

This is the sequence the whole app is built around. Use it as the one-paragraph answer if your TL asks "walk me through how this actually works".

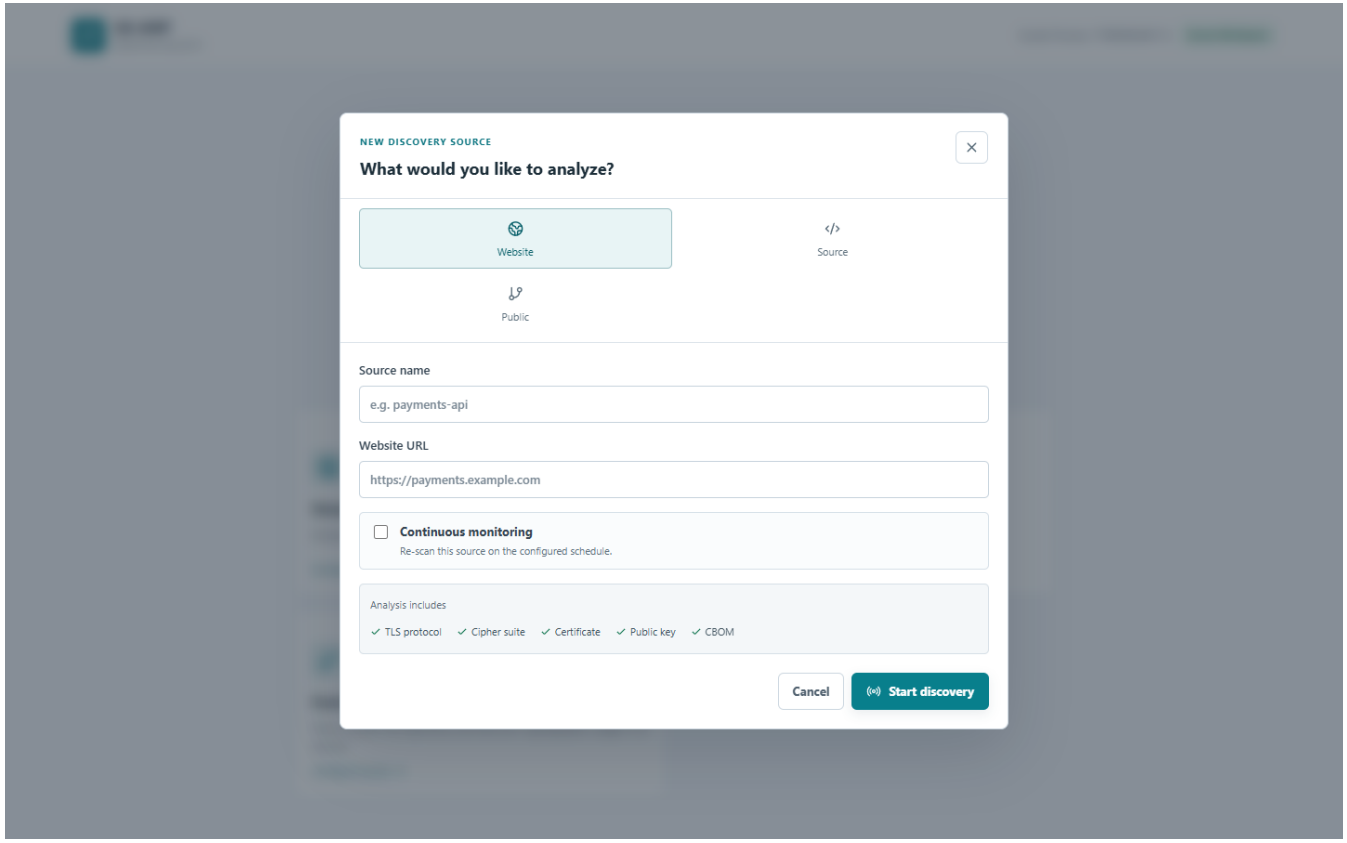
1. Add a source	User picks Website, ZIP or Public Repository and submits a target. This creates an `AssetSource` row and immediately queues a `ScanJob`.
2. Discovery runs	Website -> a real TLS handshake against the host (certificate, protocol, cipher). ZIP -> safely extracted (Zip-Slip / zip-bomb / path-traversal guarded) and statically scanned for crypto API usage. Public repo -> a real `git clone --depth 1` of the branch, then the same static scan. Every finding is written as a `CryptoAsset` with an evidence location (file path / TLS handshake).
3. CBOM is generated	The inventory for that source is serialised into a CycloneDX 1.6 document (algorithms, keys, certificates, protocols, libraries, applications, files, and the dependency edges between them).
4. Risk is scored	Every asset is re-scored against nine weighted, explainable factors (quantum vulnerability, algorithm strength, business criticality, data sensitivity, internet exposure, HNDL exposure, certificate expiry, migration complexity, dependency count). Findings above the Medium threshold are recorded.
5. PQC recommendation	For each vulnerable asset, the purpose (key exchange vs. signature vs. symmetric vs. hash) decides which PQC family is even considered - RSA/ECDH used for key exchange only ever sees ML-KEM candidates; ECDSA/RSA used for signing only ever sees ML-DSA / SLH-DSA.
6. Migration + monitoring	High-risk findings can become a Kanban migration task; if continuous monitoring was enabled on the source, it is re-scanned on schedule and any drift raises a monitoring event / alert.

Worked example (the demo data set)

The seeded demo organisation, *Acme Financial Technologies*, has a `payments-api` application using RSA-2048 for key exchange. QS-AMP: (a) flags it quantum-vulnerable, (b) recommends ML-KEM-512/768/1024 - never a signature algorithm, because the recorded purpose is key exchange, (c) shows the real FIPS 203 key/ciphertext sizes side-by-side with RSA's, and (d) offers to create a migration task moving it through Classical -> Hybrid -> PQC.

4. Onboarding & Adding a Source

The "Add source" modal is a tabbed picker. Only sources the backend can genuinely scan are offered: Website, ZIP archive, and Public Repository (a Cloud/mock-connector option existed in an earlier draft and was removed because nothing implemented it end to end).



The Add-source modal: Website tab selected, with scan options and a continuous-monitoring toggle.

Submitting the form creates the source and immediately starts a scan job. The UI then switches to a full-screen discovery animation that polls the job's progress (stage, percentage, live counters for files scanned / algorithms found / assets discovered) until it reports *completed* or *failed*.

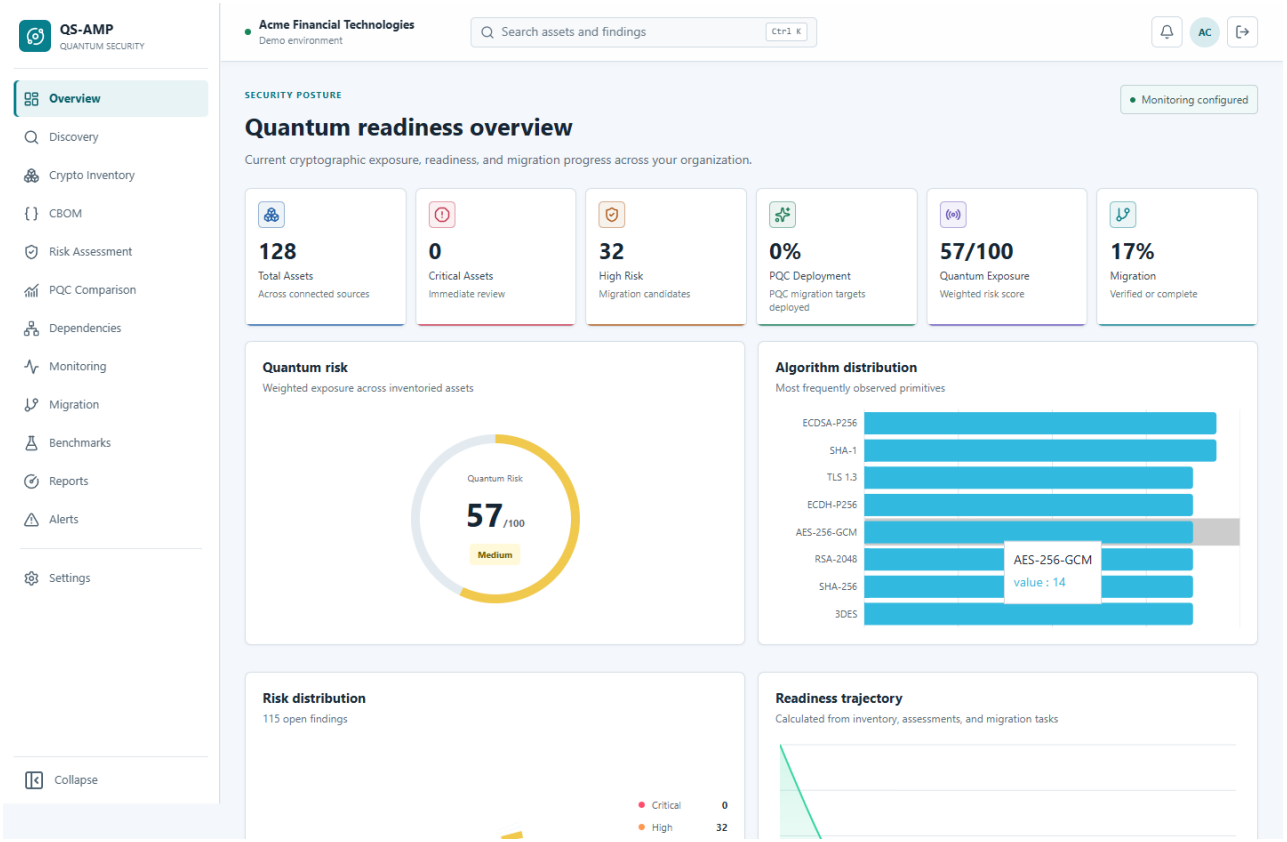
5. Module-by-Module Walkthrough

This section mirrors the left sidebar, one screen at a time. Each entry says what it's for and what backend module drives it, so you can trace any number on screen back to real code.

5.1 Overview (Dashboard)

The executive summary. Six KPI tiles (total assets, critical assets, high risk, PQC readiness, quantum exposure score, migration progress), a radial quantum-risk gauge, an algorithm-distribution bar chart, a risk donut, a readiness trend line, and a table of the most recent findings.

Powered by: `/api/dashboard/` *aggregates live counts from the Inventory, Risk and Migration modules - nothing on this page is hand-typed demo copy; every number recalculates from the database.*

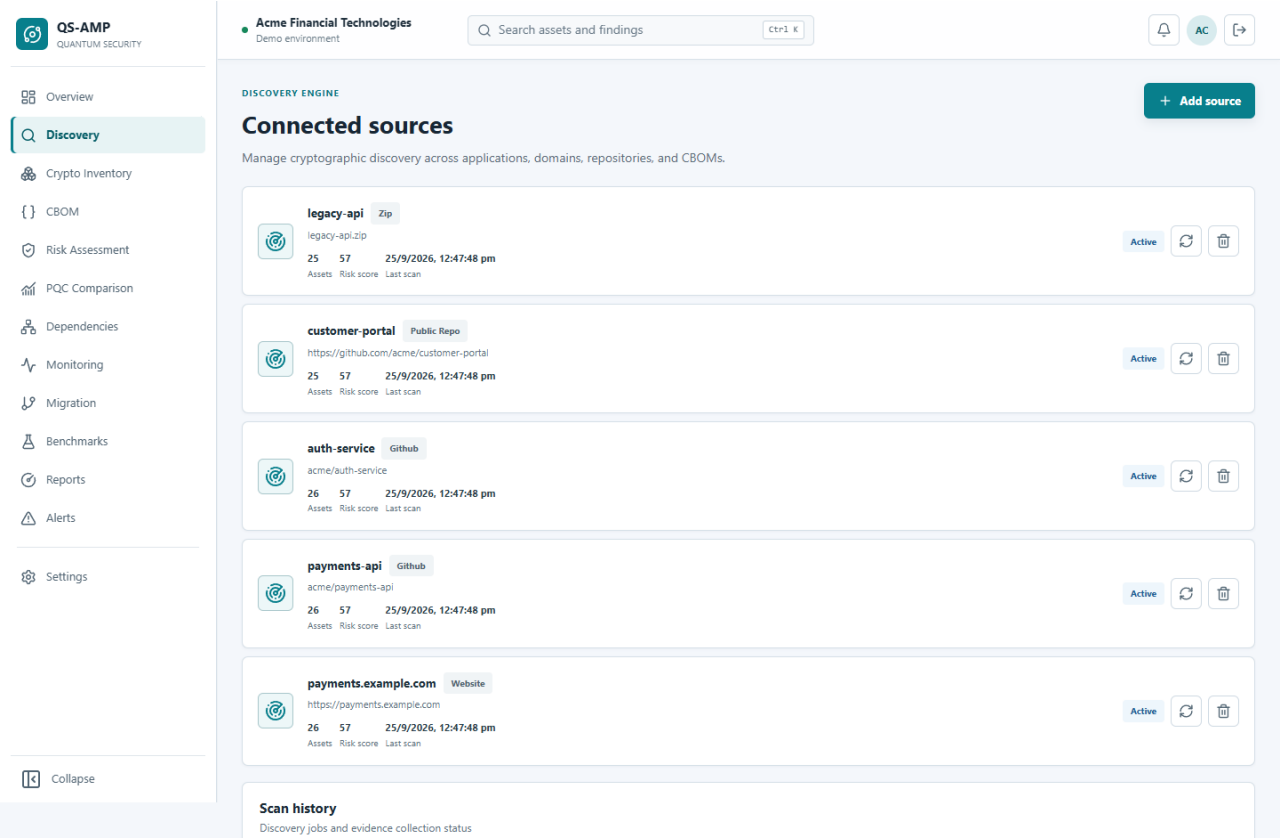


Overview dashboard for the demo organisation: 128 assets, 32 high-risk findings, 57/100 quantum exposure.

5.2 Discovery

Lists every connected source (website / ZIP / repository) with its status, asset count, risk score and last scan time, plus a scan-history table. "Scan now" re-runs discovery on demand.

Powered by: `apps.discovery` - `AssetSource`, `ScanJob`, `ScanResult` *models;* `discovery_service.py` *for the website/ZIP/repo scan implementations.*

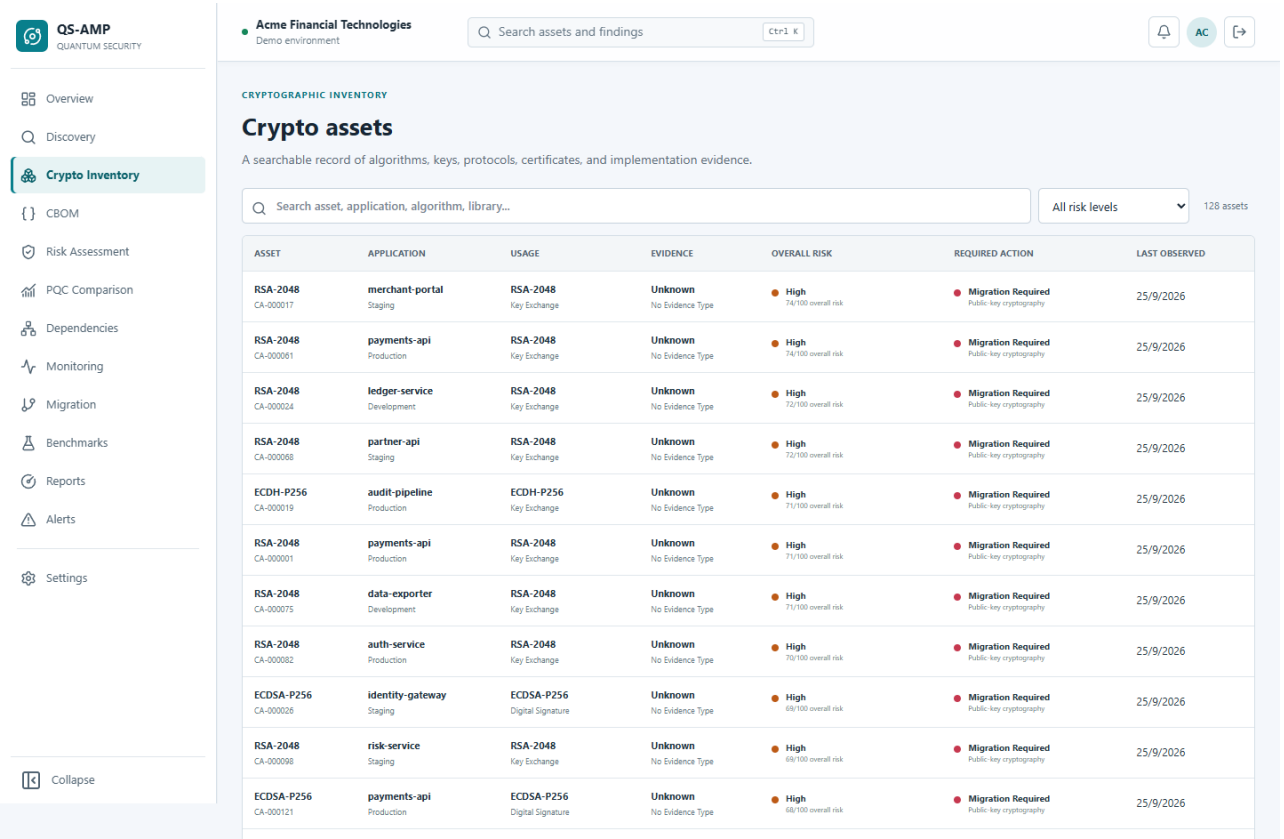


Discovery page: connected sources for the demo org, each independently re-scannable.

5.3 Crypto Inventory

The master, searchable table of every cryptographic asset ever observed - algorithm, application, purpose, key size, risk, and quantum status. Filterable by risk level; clicking a row opens a detail drawer with full evidence (file location or TLS handshake data) and a "Compare with PQC" action.

Powered by: `apps.inventory` *- the* `CryptoAsset` *model, plus* `catalog.py` *(the algorithm reference data: family, primitive, quantum status for every known algorithm, classical and post-quantum).*

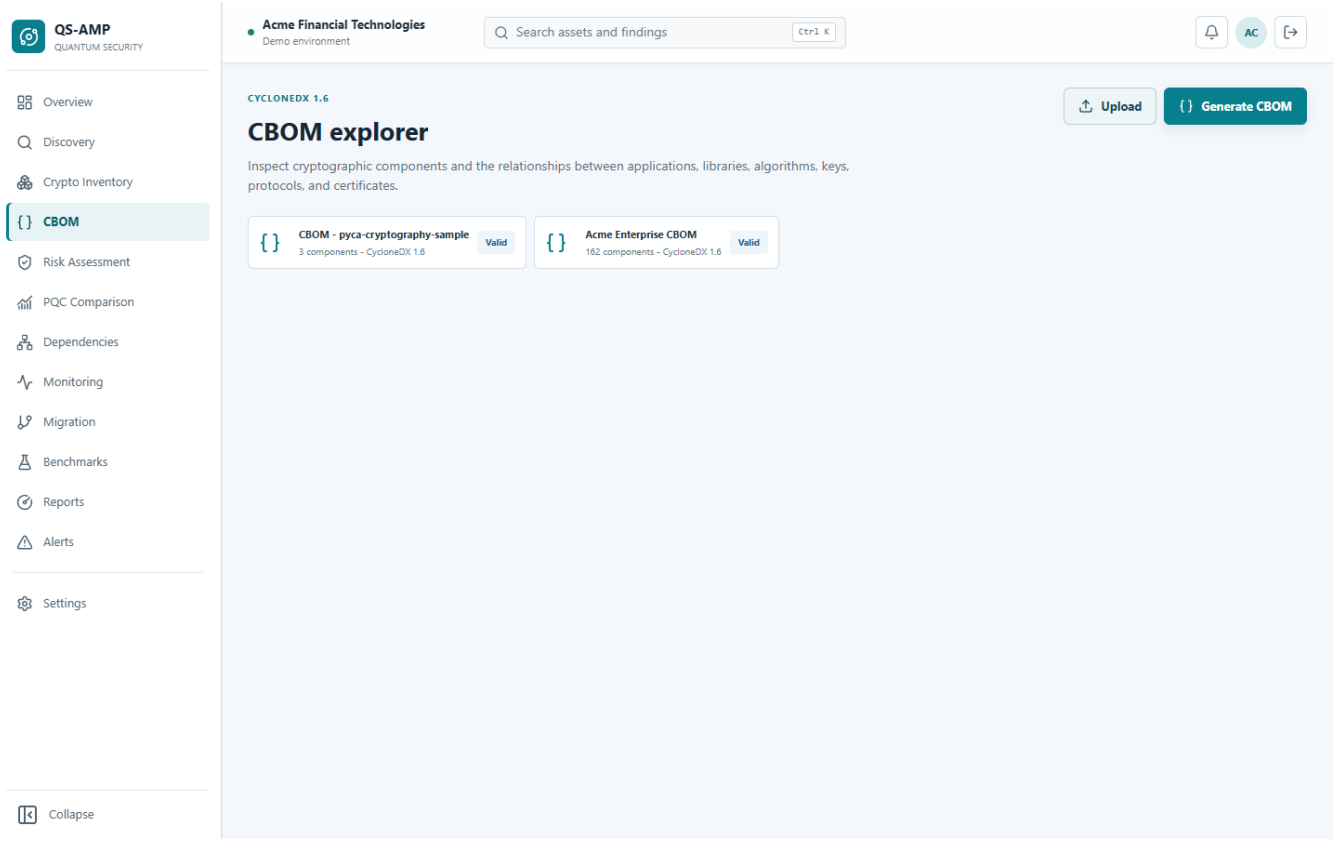


Crypto asset inventory: 128 assets, sorted by risk, each traceable to an application and evidence source.

5.4 CBOM Explorer

Generates and browses CycloneDX 1.6 Cryptography Bills of Materials - the standard machine-readable format for a crypto inventory. Supports uploading an existing CBOM (JSON or XML) as well as generating one from the live inventory, with JSON/XML export and schema validation.

Powered by: `apps.cbom` - `CBOMDocument`/`CBOMComponent`/`CBOMDependency`; `cbom_service.py` *builds and parses real CycloneDX structures (not a custom format).*

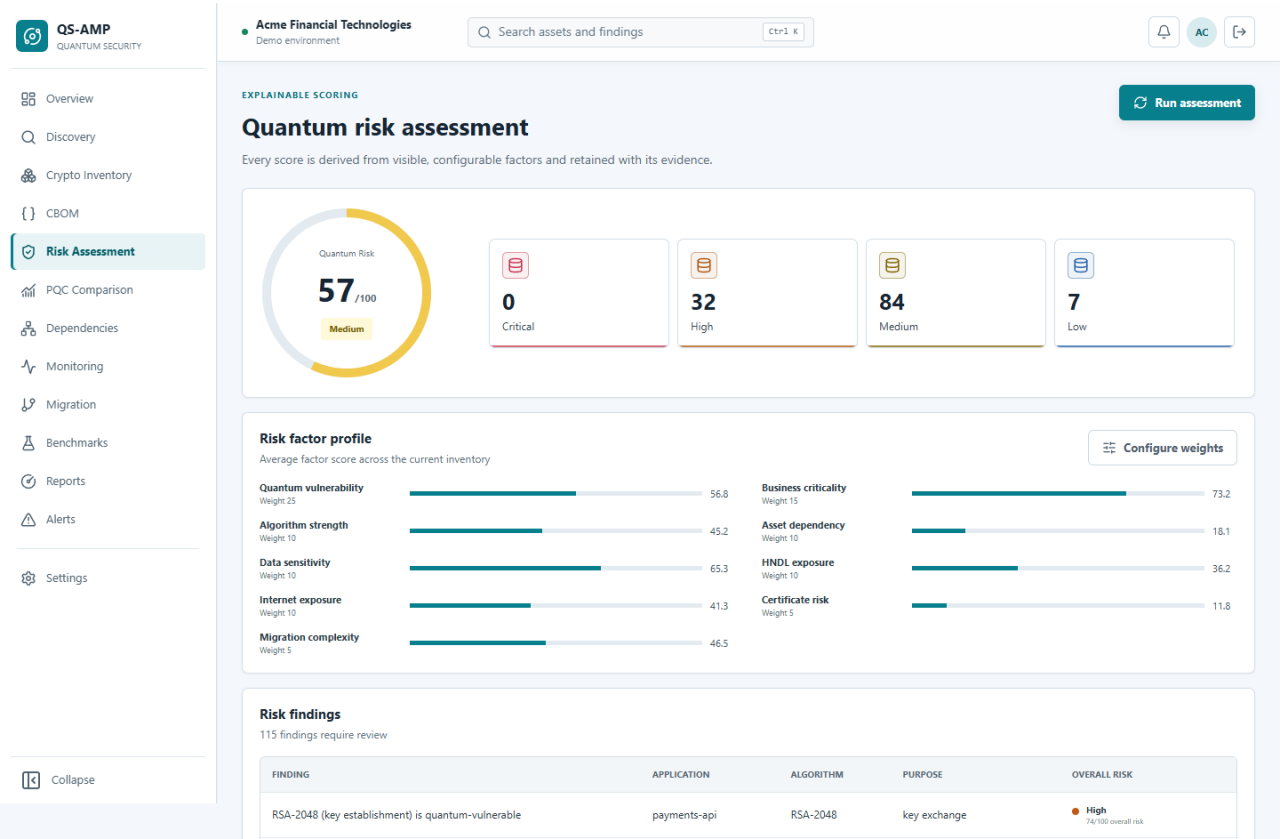


CBOM explorer: a generated organisation-wide CBOM plus one imported from a public-repository scan.

5.5 Risk Assessment

The transparent scoring engine's output. Every finding shows its final 0-100 score broken into the nine weighted factors that produced it, in plain language - never a black-box number.

Powered by: `apps.risk` - `risk_service.py`: `score_asset()` *computes and explains every factor; weights and severity thresholds are configurable per organisation under Settings.*



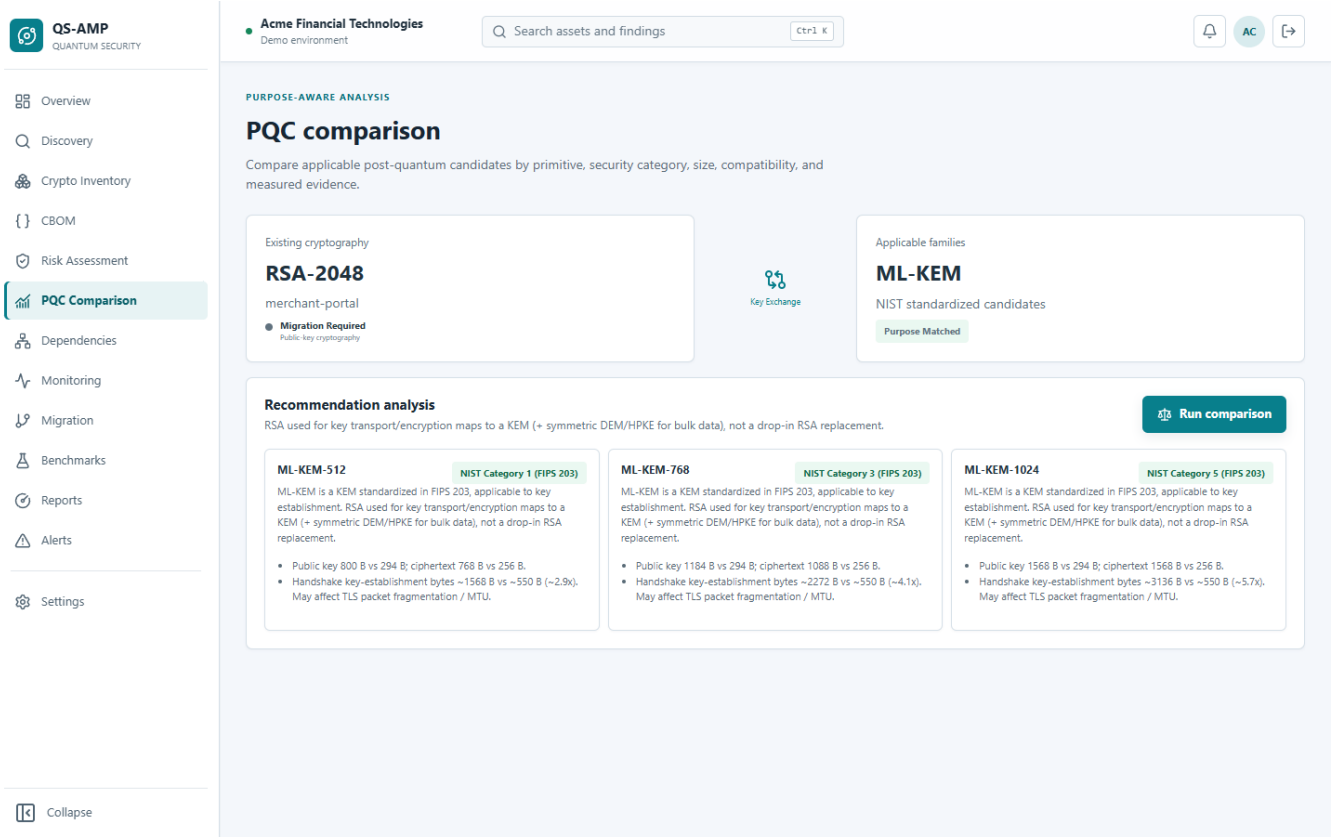
Risk assessment: overall organisation score plus the ranked list of individual findings.

5.6 PQC Comparison

The purpose-aware recommendation engine. Pick a classical asset from inventory; QS-AMP shows only the PQC families that are actually applicable to its recorded purpose, with FIPS-standard key/ciphertext sizes and (if any exist) real benchmark numbers, clearly labelled by data source.

Powered by: `apps.comparison` - `comparison_service.py`*:* `recommend_pqc()` *and* `compare_classical_vs_pqc()`*. A KEM-vs-signature mismatch is rejected server-side with an explicit error, not silently allowed.*

- Verified live: comparing ECDSA-P256 (a signature) against ML-KEM-768 (a KEM) is rejected with "incompatible_primitives" - this rule is enforced in the API, not just the UI.

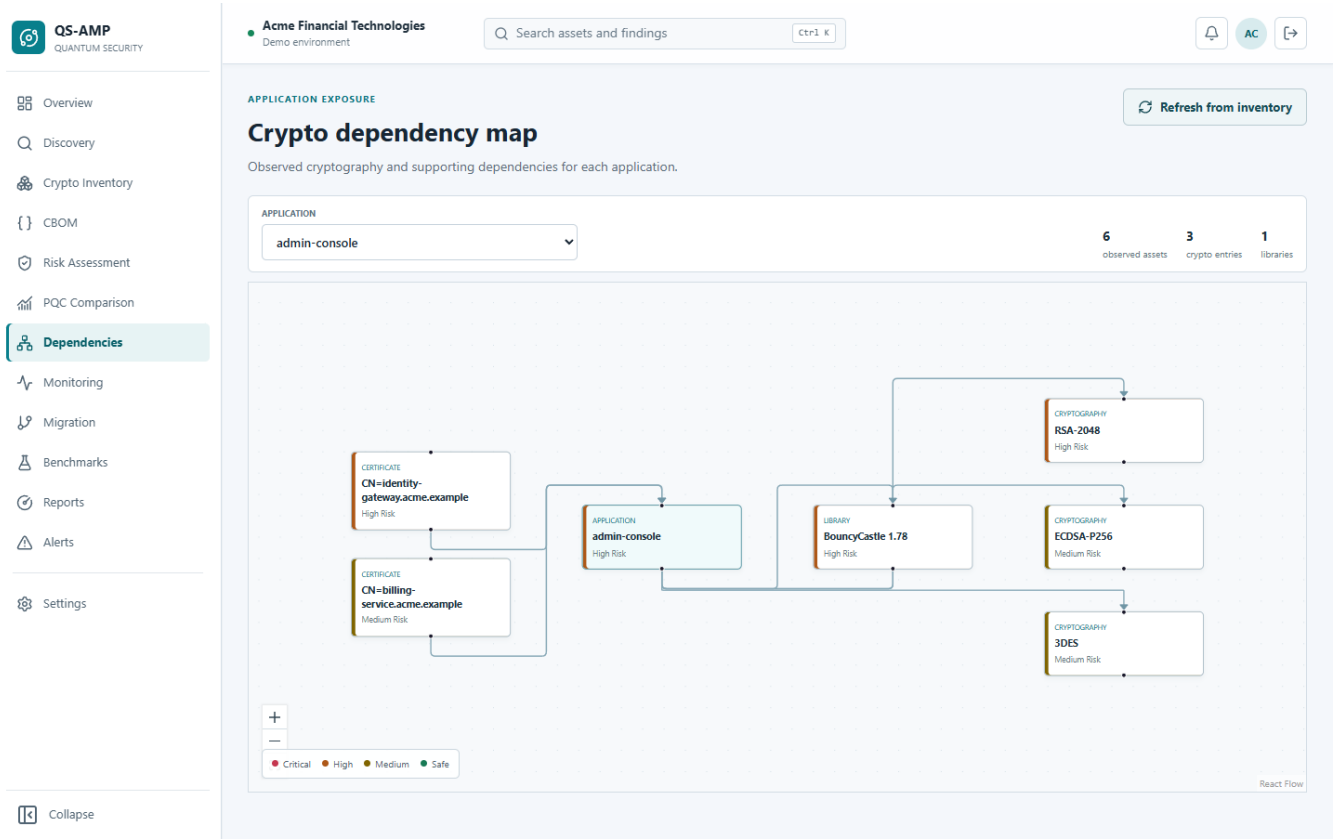


PQC comparison for a payments RSA-2048 key-exchange asset: only ML-KEM candidates are offered, with size deltas.

5.7 Dependencies (Architecture graph)

An interactive node graph (React Flow) per application: which certificates, libraries and algorithms it uses, colour-coded by risk. Lets you see the blast radius of a single weak algorithm across an application.

Powered by: `apps.inventory.architecture_service.py` *builds* `ArchitectureNode/ArchitectureEdge` *rows from the live inventory; rendered client-side with* `@xyflow/react`.

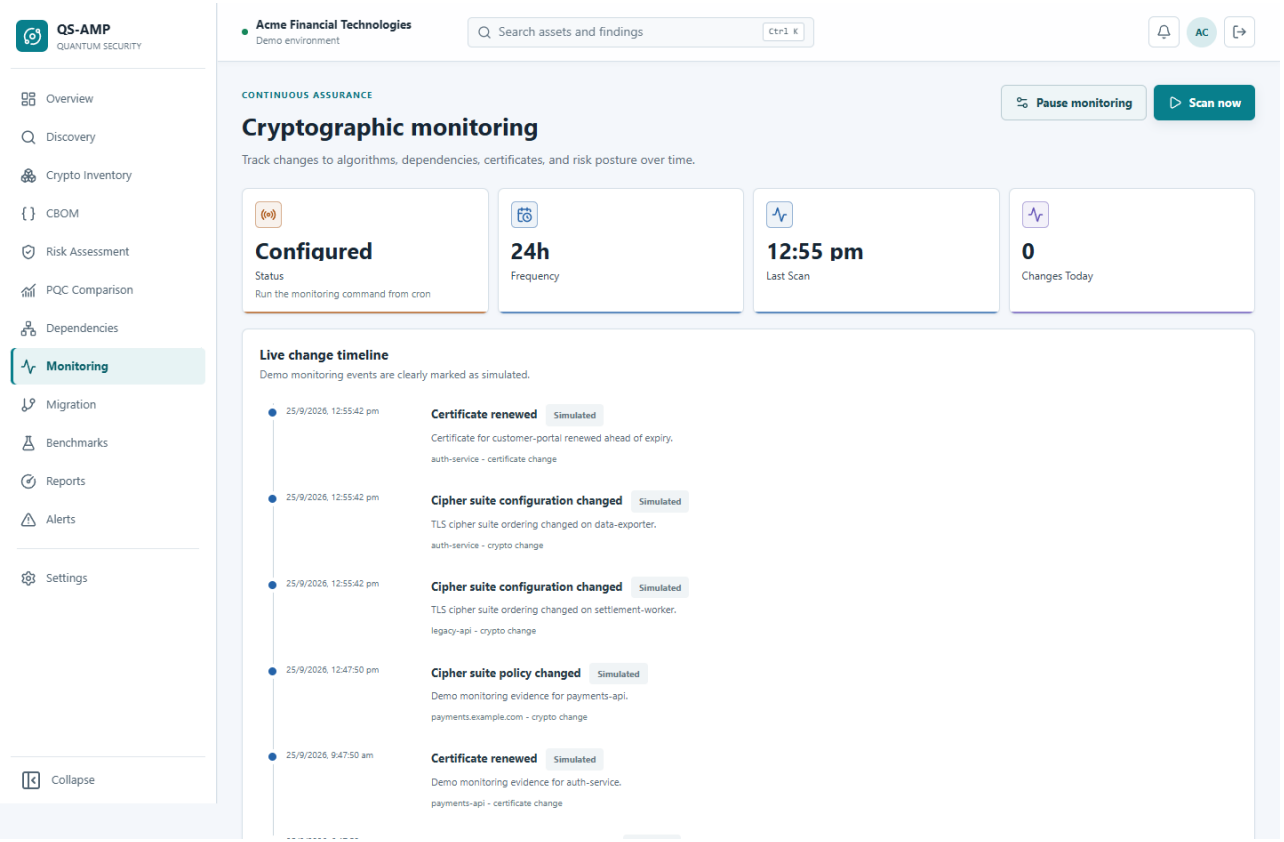


Crypto dependency map for admin-console: certificates and a BouncyCastle library feeding RSA/ECDSA/3DES usage.

5.8 Monitoring

Continuous-monitoring status and a live event timeline (certificate changes, new dependencies, new algorithms, risk changes). Sources can opt into scheduled re-scans; a demo-mode simulator produces realistic events when no real schedule has fired yet, always labelled as simulated.

Powered by: `apps.monitoring` - `MonitoringConfiguration`/`MonitoringEvent`; `monitoring_service.py` *(*`simulate_scan` *for demo events,* `real_scan`/`run_due_scans` *for the real scheduled path).*

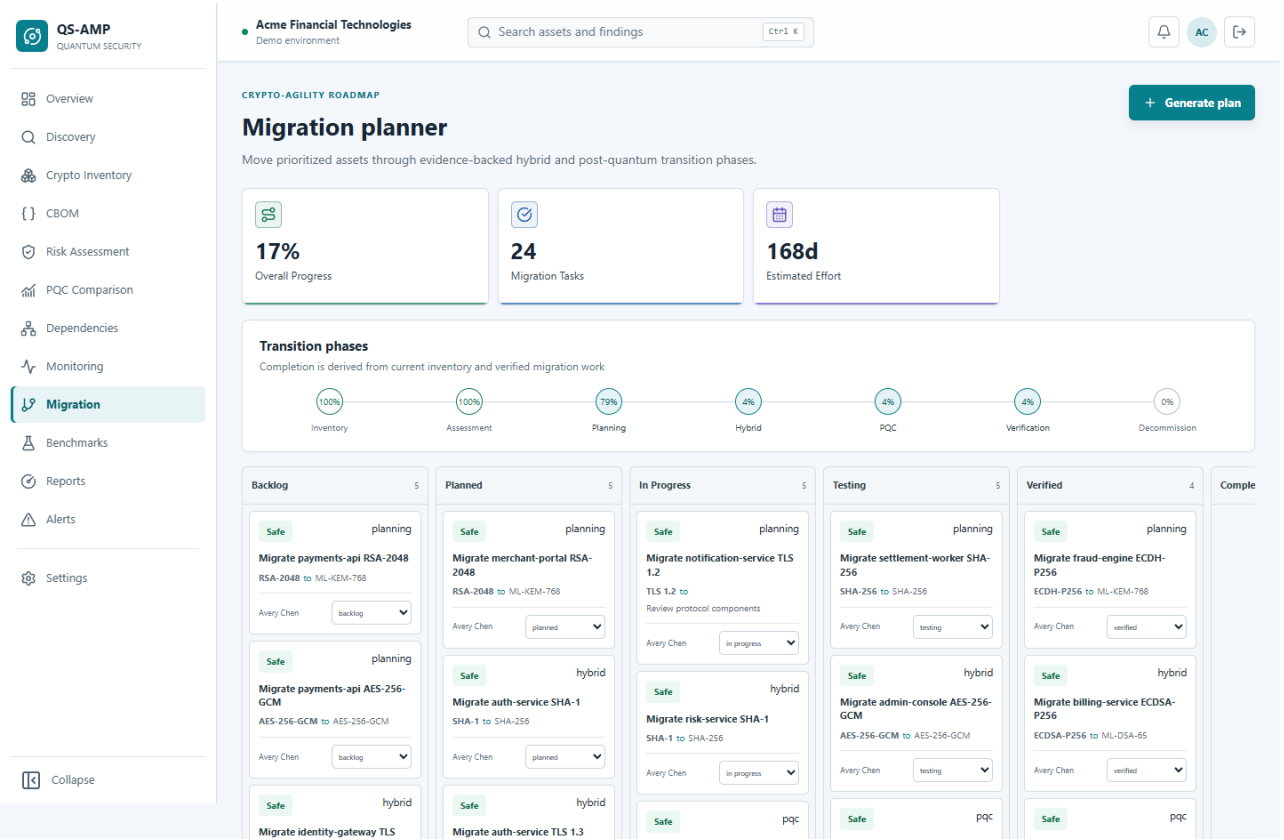


Monitoring: configuration status and the live event feed for the demo organisation.

5.9 Migration Planner

A Kanban board (Backlog -> Planned -> In Progress -> Testing -> Verified -> Completed) tracking every asset's move from its current algorithm to a PQC target, plus a 7-phase progress rail (Inventory, Assessment, Planning, Hybrid, PQC, Verification, Decommission).

Powered by: `apps.migration` - `MigrationProject/MigrationTask/MigrationDependency`. *Targets are stored as recommendation candidates, never asserted as the single correct answer.*



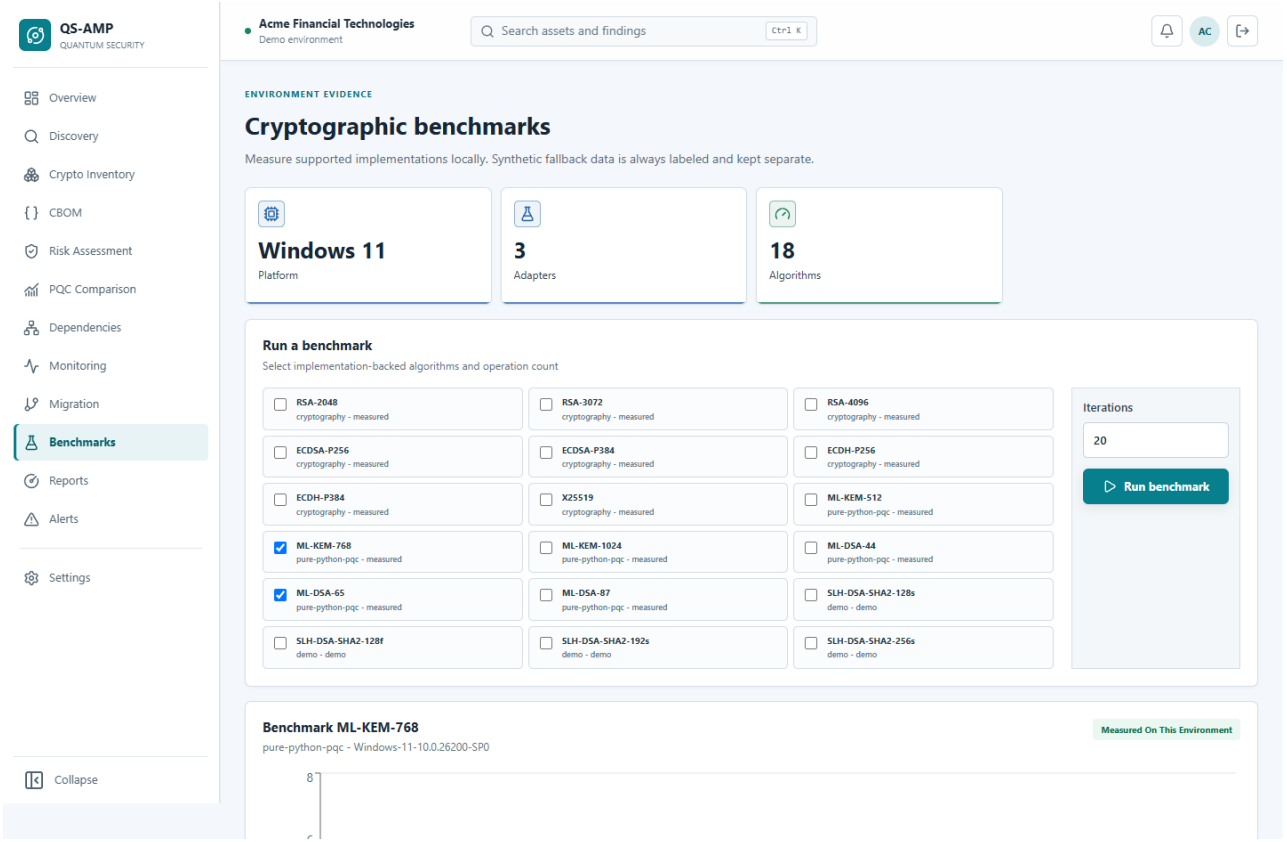
Migration planner: 24 tasks across six Kanban columns, 17% overall progress.

5.10 Benchmarks

Runs and stores real algorithm timing measurements - key generation, encapsulation/decapsulation, sign/verify - using whatever cryptographic implementation is actually available in the environment.

Powered by: `apps.benchmarks` - `BenchmarkRunner` *pattern in* `benchmark_service.py` *with pluggable adapters: pyca/cryptography (OpenSSL, classical algorithms), kyber-py/dilithium-py (pure-Python ML-KEM/ML-DSA), an optional native liboqs adapter, and a clearly labelled synthetic Demo adapter used only when nothing real is available.*

- Verified live: an ML-KEM-768 key-generation benchmark actually executed the kyber-py implementation on the host and returned a real measured timing - not a placeholder.

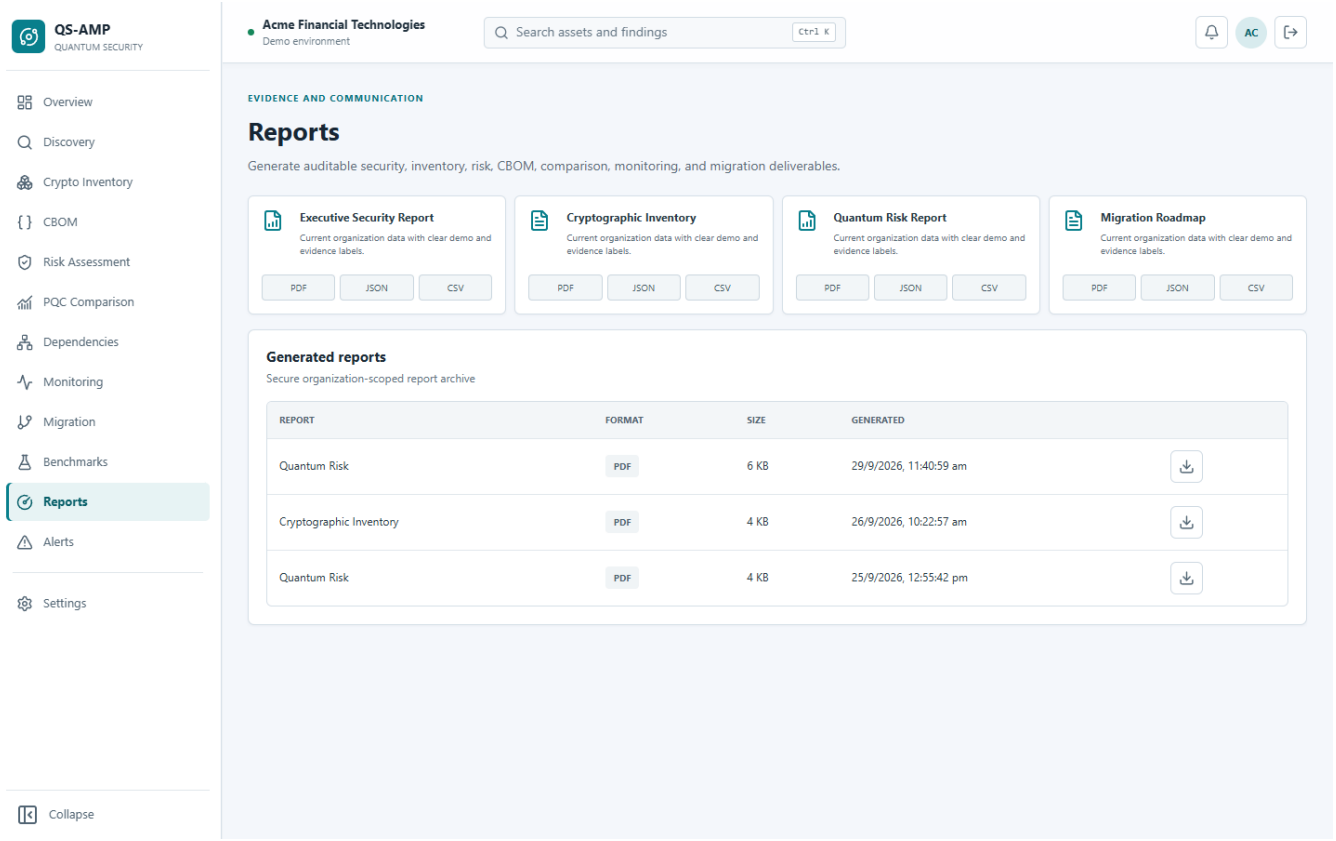


Benchmark run configuration and results, each row labelled by data source (measured vs. demo).

5.11 Reports

Generates four report types (Executive Security, Cryptographic Inventory, Quantum Risk, Migration Roadmap) as downloadable PDF, CSV or JSON files - each one a real file on disk, not a client-side print dialog.

Powered by: `apps.reports` - `Report` *model;* `report_service.py` *renders PDFs with ReportLab from the same live data every other screen reads.*

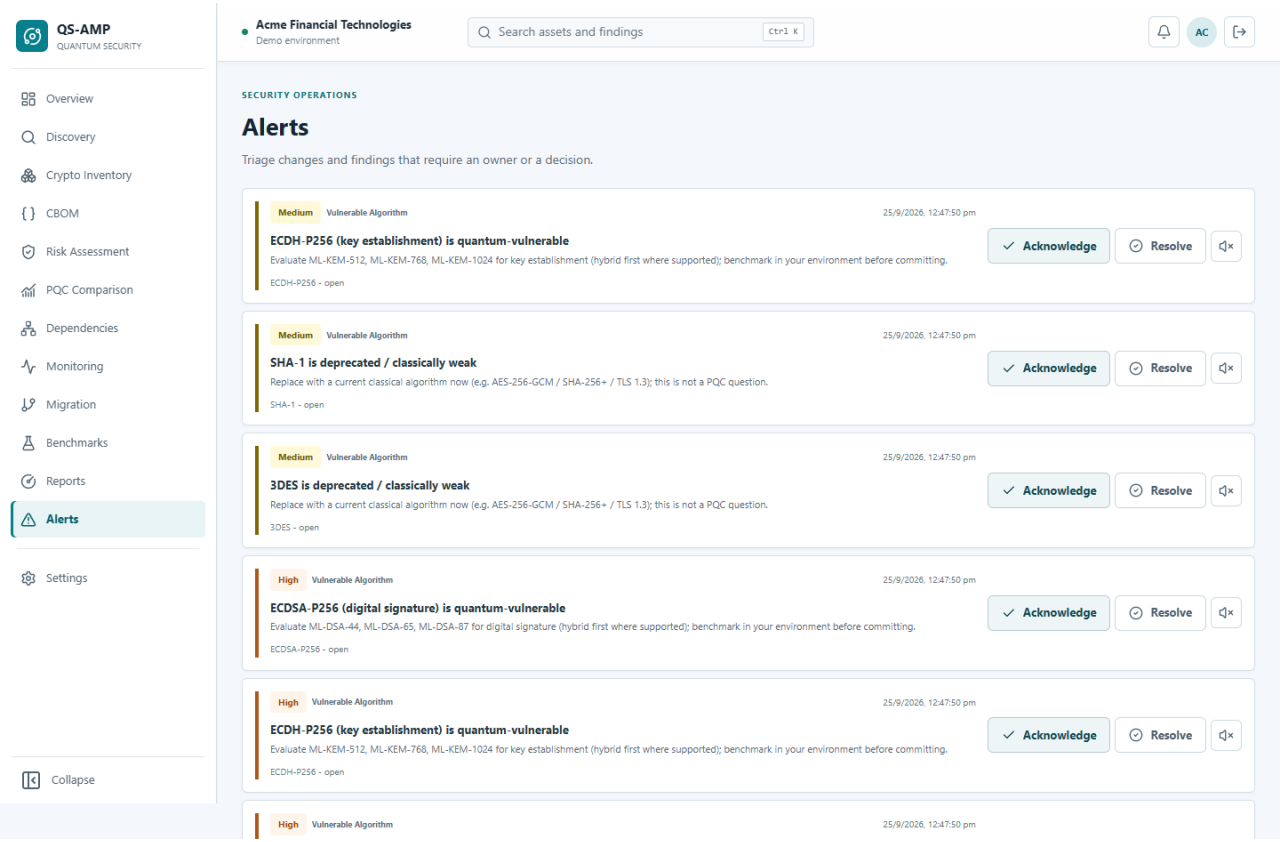


Reports library: generated PDF/CSV artifacts, each downloadable and re-fetchable later.

5.12 Alerts

A triage queue for anything that needs a human decision - new vulnerable algorithm, certificate expiring, new crypto dependency, risk increase, scan failure. Supports acknowledge / resolve / assign / mute.

Powered by: `apps.alerts` - `Alert` *model;* `alert_service.py` *creates alerts from risk re-scoring and monitoring events, with de-duplication so the same finding doesn't spam the queue.*

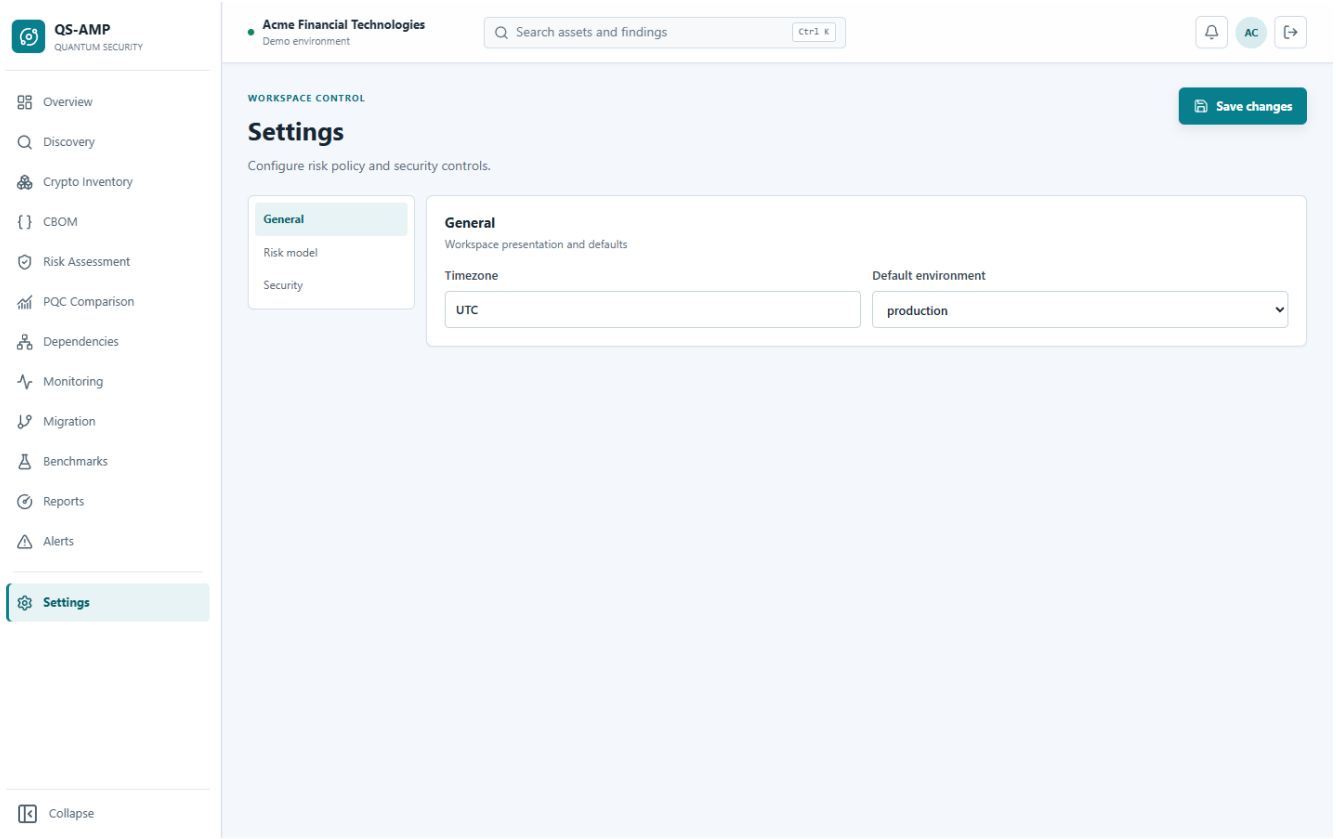


Alerts queue: open findings by severity, ready to acknowledge, resolve or assign.

5.13 Settings

Organisation-wide configuration that works even before any source exists: general settings, team roles (Owner/Admin/Security Analyst/Developer/Auditor/Viewer), risk factor weights and severity thresholds, scanning defaults, notification rules, API keys, and mock integrations (GitHub, Slack, AWS/Azure/GCP, etc.).

Powered by: `apps.organizations` *- role permissions in* `permissions.py`*, configurable defaults in* `defaults.py`*;* `apps.integrations` *for the connector cards.*



Settings: organisation, security and risk-weight configuration.

6. What Is Real vs. Simulated - Be Ready for This Question

Every TL asks this. Here is the honest breakdown, verified by exercising the running app end to end, not just reading the code.

Capability	Status
Website TLS scan	Real: opens an actual TLS handshake against the target host and reads the live certificate/cipher/protocol.
ZIP source scan	Real: safely extracts the archive (Zip-Slip / zip-bomb / path-traversal guarded) and statically greps for crypto API usage.
Public repository scan	Real: runs an actual `git clone --depth 1` of the given URL/branch, then the same static scan.
CBOM generation/import/export	Real CycloneDX 1.6 structures - validated, not a made-up JSON shape.
Risk scoring	Real, deterministic and fully explainable - not random or hard-coded per algorithm.
PQC recommendation sizes	Real specification data from FIPS 203/204/205 - not estimated.
Benchmark timings	Real measurements when a supporting library is available (OpenSSL via pyca/cryptography for classical algorithms; kyber-py/dilithium-py for ML-KEM/ML-DSA). Falls back to a clearly labelled synthetic "Demo benchmark" only when nothing real can run - the two are never mixed in the UI.
Continuous monitoring schedule	The scheduling and event model are real; in local/demo mode, events are produced by a simulator (explicitly labelled) instead of waiting for a real interval to elapse - the same code path supports real scheduled re-scans via Celery beat / a cron command.
Cloud connectors (AWS/Azure/GCP) on the Integrations page	Deliberately mocked - shows a "connection configured" state, does not call a real cloud API. This was a known, intentional MVP scope limit.
"Cloud / Infrastructure" as an addable discovery source	Removed. It existed as an option with no backing implementation and has been deleted from both the source-type list and the Add-source UI.

7. Security & Correctness Highlights

Security controls

- **Authentication:** JWT access + refresh tokens (djangorestframework-simplejwt), optional TOTP MFA (pyotp), and hashed API keys (only a SHA-256 hash is ever stored; the raw key is shown once at creation).
- **Passwords:** hashed with Django's PBKDF2 hasher; never logged or stored in plaintext.
- **Organisation isolation (multi-tenancy):** every org-owned table carries an explicit `organization` foreign key, and a shared mixin (`OrgScopedMixin`) filters every queryset by the caller's active organisation before it ever reaches a serializer.
- **Role-based permissions:** Owner / Admin / Security Analyst / Developer / Auditor / Viewer, enforced per endpoint (read vs. write roles), not just hidden in the UI.
- **File-upload safety:** ZIP uploads are checked for Zip-Slip (path traversal), decompression bombs, symlinks and disallowed executable extensions before a single byte is written to disk; uploads live under a media directory outside the served source tree.
- **Consistent error format:** every API error returns the same JSON envelope (`{"success": false, "error": {"code", "message"}}`), so the frontend never has to guess a shape.
- **Audit log:** every create/update/delete across the org-scoped modules is recorded with who, what and when.

Cryptographic correctness (the part most tools get wrong)

- The recommendation engine never treats "post-quantum" as one interchangeable bucket. A KEM (ML-KEM) is only ever offered for key-exchange/encryption uses; a signature scheme (ML-DSA/SLH-DSA) is only ever offered for signing uses. Mismatched pairs are rejected by the API itself, confirmed by direct testing.
- Claims are hedged deliberately - the app says an algorithm is "designed/standardised for post-quantum security" or "believed secure under current analysis", never "quantum-proof". This matches how NIST and the security community describe these algorithms.
- Symmetric ciphers (AES) and hash functions (SHA-2/3) are analysed separately from public-key algorithms, because Grover's algorithm affects them differently (a smaller effective security margin, not a full break) - the engine never recommends a PQC algorithm as a replacement for AES or SHA-256.

8. Quick Answers for Common Questions

Q: What is this, in one sentence?

A: A platform that scans real code/websites/repos for cryptography, scores the quantum risk of what it finds, and tells you the correct post-quantum replacement and migration plan for each specific use.

Q: Is any of this simulated or fake?

A: The scanning, inventory, CBOM, risk scoring, PQC recommendation sizes and most benchmarks are real, verified by exercising the running app. Continuous-monitoring events default to a clearly-labelled simulator until a real interval elapses, and cloud connectors are intentionally mocked in this MVP. See section 6 for the full list.

Q: Why can't I just tell it to replace RSA with a PQC algorithm automatically?

A: Because RSA is used for two different jobs - key exchange and signing - and they map to different PQC families (ML-KEM vs. ML-DSA/SLH-DSA). QS-AMP determines the recorded purpose first and only then narrows the candidate list; this is enforced server-side, not just suggested in the UI.

Q: What database does it use, and is tenant data actually isolated?

A: PostgreSQL. Every organisation-owned table has an explicit organization_id and a shared query mixin filters by it on every request, before serialization - there is no code path that returns another organisation's rows.

Q: How do I run it locally?

A: Backend: create a PostgreSQL database, copy backend/.env.example to .env and fill in the DB credentials, then python manage.py migrate && python manage.py seed_demo && python manage.py runserver. Frontend: npm install && npm run dev in frontend/, pointed at the backend's /api/ base URL via VITE_API_BASE_URL. docker-compose.yml at the repo root wires Postgres + Redis + backend + frontend together for one-command startup.

Q: What's the biggest known gap right now?

A: GitHub OAuth-based repository connection (as opposed to a public repo URL, which works today) and the mocked cloud connectors are the two places where the UI's ambition is ahead of a real backend integration. Everything else on the sidebar is backed by real, working code.